C2 Part 2: Data Representation for ASCII and Unicode

1. Bits and Bytes

- How big is a Byte?
  - Today's computers : 8-bit, 16-bit, 32-bit byte

- Byte size will determine the maximum number of possible values that can be stored.
  - 8 bits : … → 2^8 = 256 values
  - 16 bits : … → 2^{16} = 65,536 values
  - 32 bits : … → 2^{32} = 4,294,967,296 values

2. Representation of Character Sets
a. ASCII

- These two computers cannot communicate with each other because they uses different Character Representation Sets.

- 1960s – agreed a standard set of codes
- American Standard Code for Information Interchange (ASCII)
- "Ask-key"

a. ASCII

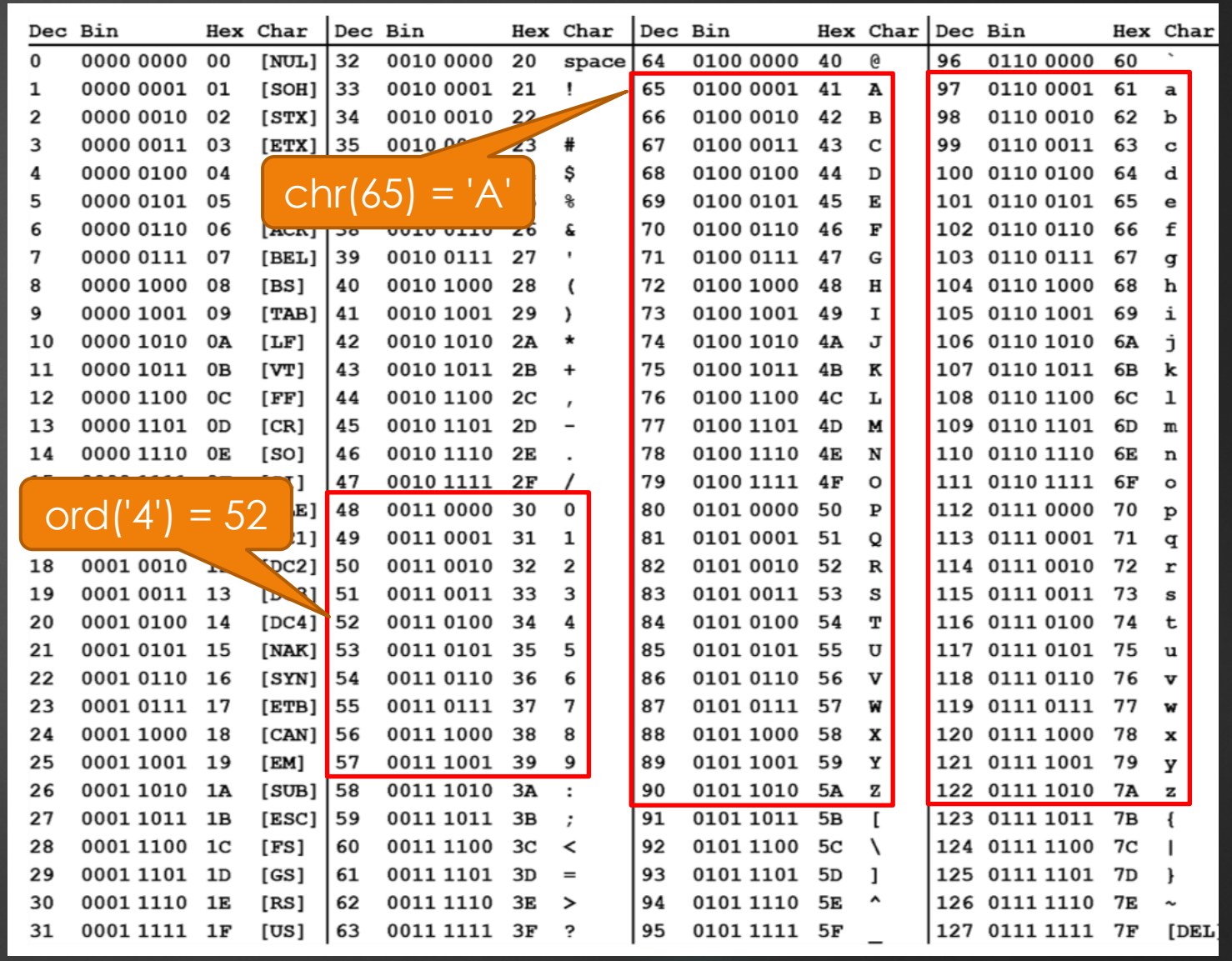
- ASCII uses a **7-bit** to represent **128** characters (95 printable characters and 33 non-printable control codes), sufficient to represent all the characters on a standard keyboard

How many keys are there in a standard keyboard?

a. ASCII

- Uses **7 bits** to represent each character
- **128 (2^7)** characters can be represented in the standard ASCII character set.

ASCII table can be found at (https://www.cs.cmu.edu/~pattis/15-1XX/common/handouts/ascii.html)



Dec	Bin	Hex	Char	Dec	Bin	Hex	Char	Dec	Bin	Hex	Char	Dec	Bin	Hex	Char
0	0000 0000	00	[NUL]	32	0010 0000	20	space	64	0100 0000	40	@	96	0110 0000	60	`
1	0000 0001	01	[SOH]	33	0010 0001	21	!	65	0100 0001	41	A	97	0110 0001	61	a
2	0000 0010	02	[STX]	34	0010 0010	22	"	66	0100 0010	42	B	98	0110 0010	62	b
3	0000 0011	03	[ETX]	35	0010 0011	23	#	67	0100 0011	43	C	99	0110 0011	63	c
4	0000 0100	04	[EOT]	36	0010 0100	24	$	68	0100 0100	44	D	100	0110 0100	64	d
5	0000 0101	05	[ENQ]	37	0010 0101	25	%	69	0100 0101	45	E	101	0110 0101	65	e
6	0000 0110	06	[ACK]	38	0010 0110	26	&	70	0100 0110	46	F	102	0110 0110	66	f
7	0000 0111	07	[BEL]	39	0010 0111	27	'	71	0100 0111	47	G	103	0110 0111	67	g
8	0000 1000	08	[BS]	40	0010 1000	28	(	72	0100 1000	48	H	104	0110 1000	68	h
9	0000 1001	09	[TAB]	41	0010 1001	29	)	73	0100 1001	49	I	105	0110 1001	69	i
10	0000 1010	0A	[LF]	42	0010 1010	2A	*	74	0100 1010	4A	J	106	0110 1010	6A	j
11	0000 1011	0B	[VT]	43	0010 1011	2B	+	75	0100 1011	4B	K	107	0110 1011	6B	k
12	0000 1100	0C	[FF]	44	0010 1100	2C	,	76	0100 1100	4C	L	108	0110 1100	6C	l
13	0000 1101	0D	[CR]	45	0010 1101	2D	-	77	0100 1101	4D	M	109	0110 1101	6D	m
14	0000 1110	0E	[SO]	46	0010 1110	2E	.	78	0100 1110	4E	N	110	0110 1110	6E	n
[illegible]				47	0010 1111	2F	/	79	0100 1111	4F	O	111	0110 1111	6F	o
[illegible]				48	0011 0000	30	0	80	0101 0000	50	P	112	0111 0000	70	p
[illegible]				49	0011 0001	31	1	81	0101 0001	51	Q	113	0111 0001	71	q
18	0001 0010	[illegible]	[DC2]	50	0011 0010	32	2	82	0101 0010	52	R	114	0111 0010	72	r
19	0001 0011	13	[illegible]	51	0011 0011	33	3	83	0101 0011	53	S	115	0111 0011	73	s
20	0001 0100	14	[DC4]	52	0011 0100	34	4	84	0101 0100	54	T	116	0111 0100	74	t
21	0001 0101	15	[NAK]	53	0011 0101	35	5	85	0101 0101	55	U	117	0111 0101	75	u
22	0001 0110	16	[SYN]	54	0011 0110	36	6	86	0101 0110	56	V	118	0111 0110	76	v
23	0001 0111	17	[ETB]	55	0011 0111	37	7	87	0101 0111	57	W	119	0111 0111	77	w
24	0001 1000	18	[CAN]	56	0011 1000	38	8	88	0101 1000	58	X	120	0111 1000	78	x
25	0001 1001	19	[EM]	57	0011 1001	39	9	89	0101 1001	59	Y	121	0111 1001	79	y
26	0001 1010	1A	[SUB]	58	0011 1010	3A	:	90	0101 1010	5A	Z	122	0111 1010	7A	z
27	0001 1011	1B	[ESC]	59	0011 1011	3B	;	91	0101 1011	5B	[	123	0111 1011	7B	{
28	0001 1100	1C	[FS]	60	0011 1100	3C	<	92	0101 1100	5C	\	124	0111 1100	7C	|
29	0001 1101	1D	[GS]	61	0011 1101	3D	=	93	0101 1101	5D	]	125	0111 1101	7D	}
30	0001 1110	1E	[RS]	62	0011 1110	3E	>	94	0101 1110	5E	^	126	0111 1110	7E	~
31	0001 1111	1F	[US]	63	0011 1111	3F	?	95	0101 1111	5F	_	127	0111 1111	7F	[DEL]

a. ASCII

- When it was standardised to have 8-bit per bytes for computer and peripherals.
- Only the first 128 was used to represent the ASCII character, the remaining 128 of the 2^8 (i.e. 256) were unused.
- The remaining 128 unused values are insufficient to represent the characters from all other languages.

2. Representation of Character Sets

b. Unicode

- Unicode was originally designed in 1980 to use **16 bit per character** to represent **65,526 (2^{16})** different characters.
- The first version of Unicode in 1991 represented **7,129** characters from **24** languages.
- By the time the 3rd version in 1999, a total of **59,501** characters from **38** languages were represented.

b. Unicode

- From 1996 onwards, Unicode is **no longer restricted to 16-bits**. This increased the capacity to represent all the existing languages' characters and even many historic scripts.
- In version 15 of the Unicode (2023), there are **304,115** characters from **161** languages being represented.

2. Representation of Character Sets

b. Unicode

- **16-bit** code (2 bytes)
- Can represent **65,526 (2^{16})** characters
- All the characters used by the world's languages

Love in different languages

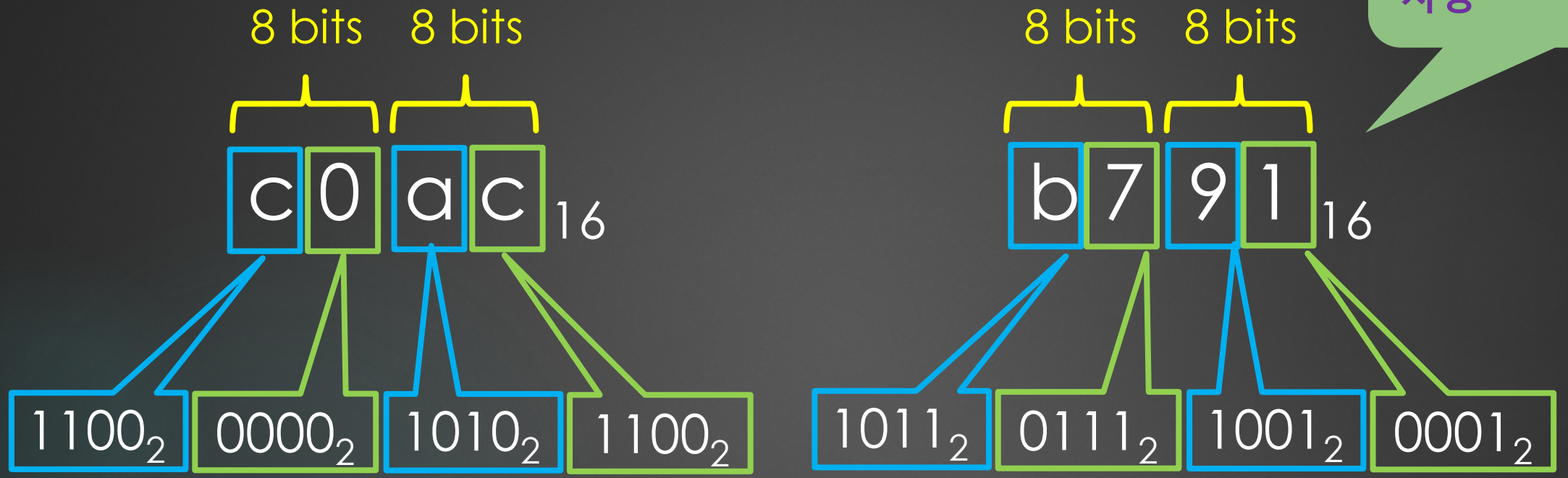
Language	Unicode	Character
Arabic	\u062d\u0628	حب
Burmese	\u1021\u1001\u103b\u1005\u103a	အချစ်
Chinese	\u7231	爱
Greek	\u03b1\u03b3\u03ac\u03c0\u03b7	αγάπη
Hindi	\u092a\u094d\u0930\u0947\u092e	प्रेम
Korean	\uc0ac\ub791	사랑
Loa	\u0eae\u0eb1\u0e81	ຮັກ
Russian	\u043b\u044e\u0431\u043b\u044e	люблю
Tamil	\u0b95\u0bbe\u0ba4\u0bb2\u0bcd	காதல்

```
>>> '\uc0ac\ub791'
'사랑'
```

Try this in Jupyter or IDLE

Unicode Converter: https://www.branah.com/unicode-converter
Google Translate: https://translate.google.com/

Unicode \uc0ac\ub791




```
>>> '\uc0ac\ub791'
'사랑'
```

11000000 10101100 10110111 10010001

2 bytes 2 bytes

3. Data Transmission: **UTF-8** Encoding

- Data are transmitted between devices using binary 0s and 1s.

  Note the following examples:

1. The ASCII and Unicode values for the dollar sign character '**$**' are the same, it is **24** in **hexadecimal**, which is **010 0100** in **7-bits binary**.

Under UTF-8 transmission, the character '**$**' can be transmitted with **one byte** by adding a '**0**' bit front.

Hence '**$**' is transmitted as **0010 0100**.

3. Data Transmission: **UTF-8** Encoding

2. The Unicode value for the English pound sign character '**£**' is **A3** in **hexadecimal**, which is **1010 0011** in **8-bit binary**.

Under UTF-8 transmission, this character needs to be transmitted using 2 bytes:

- the first byte: **110**x xxxx
- the second byte: **10**xx xxxx

Three **0**s must be added in front to represent '**£**' as a **11-bits data**, which is **000 1010 0011**.

Hence '**£**' is transmitted in the 2 btyes as **1100 0010** and **1010 0011**.

3. The Unicode value for the Euro sign character '**€**' is **20AC** in **hexadecimal**, which is **10 0000 1010 1100** in **14-bits inary**.

Under UTF-8 transmission, this character needs to be transmitted using 3 bytes:

- the first byte: **1110** xxxx
- the second byte: **10**xx xxxx
- the third byte: **10**xx xxxx

Two **0**s must be added in front to represent '**€** ' as a **16-bits data**, which is **0010 0000 1010 1100**.

Hence 3 btyes transmitted are **1110 0010**, **1000 0010** and **1010 1100**.

3. Data Transmission: **UTF-8** Encoding

When the characters are being transmitted together with 0s and 1s, it will look like as follows:

00100100110000101010001111100010100000010101101100

Would you be able to distinguish and identify the transmitted character(s)?

3. Data Transmission: **UTF-8** Encoding

Let's group them as **8-bit** (i.e. 1 byte) each:

1st byte: **0010 0100**
2nd byte: **1100 0010**
3rd byte: **1010 0011**
4th byte: **1110 0010**
5th byte: **1000 0010**
6th byte: **1010 1100**

3. Data Transmission: **UTF-8** Encoding

Let's group them as **8-bit** (i.e. 1 byte) each:

1st byte: **0010 0100**
2nd byte: **1100 0010**
3rd byte: **1010 0011**
4th byte: **1110 0010**
5th byte: **1000 0010**
6th byte: **1010 1100**

Since the 1st byte starts with a **0**, so it represents one character.

After removing the first **0**, the character has the Unicode value of **010 0100** in binary.
(i.e. 24 in hexadecimal)

3. Data Transmission: **UTF-8** Encoding

Let's group them as **8-bit** (i.e. 1 byte) each:

1st byte: **0010 0100**
2nd byte: **1100 0010**
3rd byte: **1010 0011**
4th byte: **1110 0010**
5th byte: **1000 0010**
6th byte: **1010 1100**

Since the 2nd byte starts with a **110**, so the 2nd and 3rd bytes represents one character.

After removing the **110** in the 2nd byte and the **10** in the 3rd byte, the character has the Unicode value of **00010 100011** in binary, which is **1010 0011** in binary.
(i.e. A3 in hexadecimal)

3. Data Transmission: **UTF-8** Encoding

Let's group them as **8-bit** (i.e. 1 byte) each:

1st byte: **0010 0100**
2nd byte: **1100 0010**
3rd byte: **1010 0011**
4th byte: **1110 0010**
5th byte: **1000 0010**
6th byte: **1010 1100**

Since the 4th byte starts with a **1110**, so the 4th, 5th and 6th bytes represents one character.

After removing the **1110** in the 4th byte and the **10** in the 5th and 6th bytes, the character has the Unicode value of **010 00 0010 10 1100** in binary, which is **10 0000 1010 1100** in binary. (i.e. 20AC in hexadecimal)

3. Data Transmission: **UTF-8** Encoding

Currently, the UTF-8 can transmit up to 6 bytes:

- the first byte: **1111 110**x
- the second byte: **10**xx xxxx
- the third byte: **10**xx xxxx
- the forth byte: **10**xx xxxx
- the fifth byte: **10**xx xxxx
- the sixth byte: **10**xx xxxx

Hence the maximum number of bits for a data is 31 bits.

This can be used to represent up to 2^31, i.e. more than 2,000 million, different Unicode characters.